



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

1 - “Introduction to Computer Security”

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

1 - “Introduction to Computer Security”

This block of slides **does not refer**
to a specific chapter in the book.

It is a “gentle introduction”

Gabriele D'Angelo <g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

A few **basic principles** of computer security:

- principle 1: "*non-existence of secure systems*"
- principle 1 (**corollary**): "*systems complexity*"
- principle 2: "*entities composing a system*"
- principle 3: "*security = knowledge*"

Security of a **cryptosystem** (i.e., *security through obscurity*)

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

- **The software is often far from perfect.** It is full of **implementation bugs** and **design errors**. It is the same for **communication protocols**
- **The “unbreakable system” is only a myth.** Just like the impenetrable bank vault or the unsinkable boat (e.g., *RMS Titanic*)

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Un attaccante attacca un sistema se c'è abbastanza valore che ne vale la pena l'attacco

SECURE SYSTEMS DO NOT EXIST



Security is costly: so there are various level of security

The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

È molto difficile valutare correttamente il livello di sicurezza necessario per un sistema.

Le vulnerabilità possono essere scoperte (da una persona pagata o criminale)

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Quanto sono disposto ad investire per trovare/risolvere le vulnerabilità del mio sistema?

SECURE SYSTEMS DO NOT EXIST

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

What are we protecting?

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

Capire chi è il nemico
è importante per
comprendere il livello
di sicurezza adeguato.

What are we protecting? From **who**?

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURE SYSTEMS DO NOT EXIST

- The **security level** of a given system is determined by the **amount of time** necessary to break the system, the **amount of money** (or resources) and the **probability of success**

Quali sono le
motivazioni di un
possibile attacco?

What are we protecting? From **who?** **Why?**

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Pensare in termine di valore perchè è il modo corretto
per valutare il livello di sicurezza del sistema

SECURITY EXIST

- The **security level** is the **amount of** **time** necessary to compromise the system (or resources) and the **cost** (or **money**)
- Most times the adversary is rational
- Most times the adversary has an economic goal (that is... \$\$\$)
- This is not always true when the adversary is a government

What are we protecting? From who? Why?

Principle 1: on “Secure Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- This is an “informal way” for defining a
THREAT MODEL
- Threat modeling is a structured process
used in cybersecurity to identify and
analyze potential threats and
vulnerabilities in a system or application

DO NOT EXIST

Definendo il Threat Model, sono in grado di definire il livello di sicurezza di cui il sistema ha bisogno

determined by the **amount of**

the **amount of money** (or

success

What are we protecting? From who? Why?

er: ← catena di vulnerabilità per acquisire superprivilegi (es: vuln. su whatsapp e kernel)

Up to
\$2,500,000

Vulnerability Broker:

vulnerabilità scoperte da ricercatori e vende le vulnerabilità (entità che acquistano e vendono informazioni su falle software (di solito, zero-days) da ricercatori di sicurezza, agendo come intermediari.)

FCP: Full Chain with Persistence
RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass

- iOS
- Android
- Any OS

Quando si riavvia il dispositivo, il sistema rimane compromesso.

codice
automatica
mente
eseguito
senza
interazione
dell'utente

Prezzo pagato
da Vul. Broker
per acquistare
la vulnerabilità
da ricercatori

** All payouts are subject to change or cancellation without notice. All trademarks are the property of their respective owners.*

2019/09 © zerodium.com

ZERODIUM Payouts for Mobiles*

Up to
\$2,500,000

Up to
\$2,000,000

Up to
\$1,500,000

Up to
\$1,000,000

Up to
\$500,000

Up to
\$200,000

Up to
\$100,000

FCP: Full Chain with Persistence
RCE: Remote Code Execution
LPE: Local Privilege Escalation
SBX: Sandbox Escape or Bypass

■ iOS
■ Android
■ Any OS

While RCE allows attackers to execute commands from a remote location to a device, LPE allows a low-privileged user to gain administrative or root-level access.

A full-chain vulnerability on Mobile is a sophisticated cyberattack that links together multiple individual software flaws, often zero-day vulnerabilities, to achieve complete, unauthorized control over a device. While a single vulnerability might only cause an app to crash, a full chain links, for example, a browser exploit, a sandbox escape, and a kernel privilege escalation to move from an external attack to full "root" or kernel-level control.

Is not a full-chain vulnerability

WhatsApp RCE example:

CVE-2022-36934

Exploit Whatsapp to be able to run unauth code on the device and then doing something on the device itself (maybe there is something after that maybe another privilege escalation)



* All payouts are subject to change or cancellation without notice. All trademarks are the property of their respective owners.

2019/09 © zerodium.com

Idea:
Exploit a
Vulnerability in the
Application and then
exploit a Kernel
Vulnerability to take
control of Root
Privileges



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ZeroDium ✓ @ZeroDium · Jun 1

We're looking for #0day exploits affecting Pidgin on Windows and Linux.

Bounty: \$100,000

Read more: zerodium.com/temporary.html



9



129



213



Gary Kramlich

@rw_grim

Most of the time, the attacker is rational

The dev can introduce the vuln. and then selling them

A lot of software products rely on open-source libraries

Replying to @ZeroDium

This really shows the sad state of funding in open source software.. I managed to find 25k USD last year to work on @impidgin full time, but if you can find a security exploit in my and other's non paid work you can make 4 times as much as I did... 🤔

9:59 PM · Jun 1, 2021 · Twitter Web App

Idea Cloud Computing:
Exploit App Vulnerability, Exploit
Virtual OS Kernel Vulnerability
and Exploit Virtualization
Software Vulnerability (to escape
the VM) to arrive on the HW or
OS of the Machine

Vuln. Broker are against the Fix of the
Vulnerability (the Manufacture wants to fix it)

- 1) Sell the Vuln to Manufacture
(Participate to Bug Bounty Program: get
way less money than from Vuln. Brokers)
- 2) Sell the Vuln to Vuln. Broker

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- (**Empirical rule**) **Increasing the complexity** of a systems leads to an insecure system
- **Linear increase** of complexity means **super-linear increase** of insecurity

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Do you want to build a “**reasonably secure**” system?

Reduce the Complexity
of the System

KISS RULE:

- **Keep It Simple and Stupid**
- *Keep It Simple, Stupid*
- *Keep It Short and Stupid*

If you don't understand the
whole system, it is very difficult to
have that system secure

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Does this mean that the systems to be secured can **not** be **complex** or **complicated**?

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- Does this mean that the systems to be secured can **not** be **complex** or **complicated**?

NO!

Principle 1: on “Complexity of Systems”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

MORE COMPLEXITY → LESS SECURITY

- It is the most important part of the whole system (example: Authentication of the device to use it)
 - The **protection system** must be “*as simple as possible*”. If possible...
“*very simple*”
- Unnecessary complexity** can lead to {**design, implementation, usage**} errors
- Each **error** is a potential “**vulnerability**”

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Zones of the system where we can have a Vulnerability

(MOST) SYSTEMS ARE COMPOSED OF 3 TYPES OF ENTITIES

- Software
- Hardware (physical security is guaranteed if bad guy isn't able to put hands on the HW)
- Humanware People inside the system

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

- It is a critical part of the system with a **large exposed attack surface**
- remember that... **often other parts of the system are “easier to attack”**



- Many, very complex programs
- Each program made of thousands or millions of lines of source code
(i.e. computer instructions)

- It is a critical part of the system with a **large** exposed **attack surface**
- remember that... **often other parts of the system are “easier to attack”**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

Air-Gapped Computer: This refers to a computer system that is physically isolated from all insecure networks, including the internet. The name implies there is a "gap of air" between the machine and any network, offering maximum security against remote, network-based, or malware attacks.

- It is a critical part of the system with a large **exposed attack**

That can be **accessed remotely** (i.e. from the Internet, from a wireless network or from a user interface)

(example: cars)

the system are “**easier to**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

doing a well attack surface
analysis isn't so easy

- It is a critical part of the system with a **large exposed attack**

surface

tech or not (example: a paper document containing information about a company is considered a significant part of an organization's physical attack surface)

- remember ***Everything*** that can be used by an attacker to violate the system are “**easier to attack**”

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SOFTWARE

- It is a critical part of the system with a **large exposed attack surface**
- remember that... **often other parts of the system are “easier to attack”**

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Attacking the software often
requires some technical skills...

attacking the human in the
system requires skills that are not

about computing (psychological skills.
Example of attack: phishing)

SOFTWARE

system with a large exposed attack

- remember that... often other parts of the system are “easier to attack”

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HARDWARE

When firmware cannot be updated to remediate vulnerabilities, organizations must shift from patching to mitigation by removing the hardware or reducing its attack surface to manage risks. Firmware vulnerabilities are high-impact because they operate below the operating system level, often allowing for persistent, stealthy access.

- All the modern hardware is composed of **hardware AND software** (that is called **firmware**)
- The hardware can be **tampered** by an attacker
- Even the **physical security** of systems and devices (e.g. to deny **unauthorized access**) is hard to guarantee

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HARDWARE

- All the modern hardware is composed of **hardware AND software** (that is called **firmware**)

- This means that an attacker can modify the hardware or act on the software (e.g. flashing
- a modified firmware)

if the firmware is updatable and not well secured, the attacker can install a firmware with a backdoor

acker

nd devices (e.g. to deny

tee

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Definition: “interfere with (something) in order to cause damage or make unauthorized alterations”

ARE

composed of **hardware AND software**

physical manipulation of a device's components, circuitry, or logic with malicious intent. Tampering requires the attacker to have physical access with the hardware.

- The hardware can be **tampered** by an attacker
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

How to know if an HW have been tampered? To guarantee low-tampering HW, you need to guarantee the physical security of the device (because if an HW has been tampered, it is very difficult to know it)

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Modern CPUs incorporate hardware-based anti-tamper mechanisms, such as cryptographic firmware signing, to avoid unauthorized modifications.

HARDWARE

The Physical security of an HW must be guaranteed across the full hardware lifecycle, from initial purchase to final disposal (because if control were lost even for just a few seconds, it could be tampered). We do this because it is very difficult to detect if the HW it was tempered

The “evil maid attack” is an example of unauthorized alteration of personal devices

composed of **hardware AND software**

- The hardware is **tampered** by an attacker
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Possible to put a backdoor in a binary firmware without knowing the source code (it is easy to place it in the middle of the firmware)

HARDWARE

It is an attack that exploits temporary physical access to compromise a device so that the attacker can control it remotely (by inserting a backdoor into the device's firmware).

The “evil maid attack” is an example of **unauthorized alteration of personal devices**

Evil Maid Attack Demo

A “nice demo”: the time required to flash the tampered firmware is very limited

- The hardware can be **tampered** by an attacker
- Even the **physical security** of systems and devices (**e.g. to deny unauthorized access**) is hard to guarantee

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HUMANWARE

- The “**human in the loop**” is often the **weakest link in the system** (e.g. **social engineering**)
- For **many reasons**: lack of knowledge, overload, fatigue, bad design, goodwill, lack of attention... stupidity (*in both **users** and **system administrators***)

Principle 2: on “Entities

Social engineering is widely recognized as the most common and effective method for cyberattacks in the history of cybersecurity. According to industry statistics, social engineering techniques are involved in 98% of cyberattacks, making the “human element” the weakest link in security.

Definition: “^{inganno}the use of deception to manipulate ^{to push him/her}individuals into divulging confidential or ^(that goes against cybersecurity policies)personal information that may be used for fraudulent purposes” (e.g. phishing emails)

- The “**human in the loop**” is often the **weakest link** in the system (e.g. **social engineering**)

- For **many reasons**: lack of knowledge, ^{sovraccarico (può essere indotto dall'attaccante)}overload, fatigue, bad ^{of tools}design, goodwill, lack of attention... stupidity (in both **users** and **system administrators**)

In such cases, people tend to cut corners by failing to follow cybersecurity policies and this makes the system vulnerable

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HUMANWARE

Correct way to mitigate this problem is through procedures (properly designed) to identify if I'm talking with a colleague or impostor

- The “**human in the loop**”
(e.g. **social engineering**)

buona volontà
Is “**goodwill**” a **problem**?

(example: faking to be a colleague and ask for confidential info)

Yes, when abused by attackers

- For **many reasons**: lack of knowledge, overload, fatigue, bad design, **goodwill**, lack of attention... stupidity (*in both **users** and **system administrators***)

We are stupid when we don't understand something

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Operational Security (OPSEC) is the process of protecting sensitive information by looking at your own operations through the eyes of an attacker.

SECURITY AS A PROCESS

Cybersecurity isn't a product

- Securing a system is a [**never ending**] **process** and **not a product**

(Cybersecurity is a day-by-day investment)

- **Reasonably secure system** requires **an endless work** on the system and **education**

secure

- 1 • The system that today is “secure”, tomorrow will not be. **Everyday news faults are discovered**

- 2 • The **lack of updates** leads to **fragile** and **insecure** systems (system can be compromised)

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SysAdmin focus on Updates (when to do it, how, etc..)

EVEN UPDATING IS NOT EASY...

- Are the updates **available** for all your devices? (e.g. **smartphone**)
- In other words, **is your devices supported? Until when?**

→ if the device is no longer supported, the device is vulnerable if they discover a vulnerability and you cannot update it

Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

EVEN UPDATING IS NOT EASY...

- Are the updates **available** for all your devices? (e.g. **smartphone**)
- In other words, **is your devices supported? Until when?**

Based on
Company
Context



Update Policy

If the updates are available...

The tech guy need to define the Update Policy

- **what** update to apply? (**All of them...** are you sure of that?) Choose which part of the update to do
- **when** do you apply updates? (During the **night**? In “**production**”?)
- **why**? (Is that for **security** or to add **new functionalities**?)
- **how**? (What is the best way to apply the update? Automatic or **Unattended**?)

if the vuln. is public known, update ASAP, considering the time (example: customer peeks, etc..)



Principle 2: on “Entities composing a system”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

EVEN UPDATING IS NOT EASY...

- The **updating process** is always a dangerous operation
 - for example: fixing bugs can introduce new bugs, instability, side-effects... due to complexity of the system we are dealing with
- What we need is a **costs/benefits evaluation**
 - that is **costly** (e.g. time consuming)
 - it is often based on **partial information** to take decision on a specific update (i.e. often there is a lack of detailed information on the **updates content**)

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

Trust is even more
important than Knowledge

- Real security is based on **deep knowledge** (of the system to be protected)
- What are the **effects** of this principle **on software**?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **deep** knowledge (the specifications of the system are **OPEN / CLOSED** be protected)
- What are the **effects** of this principle on software?
 - **Open Systems vs. Closed Systems**
 - **Open Source vs. Closed Source**

I really prefer **OPEN** specifications (e.g. for interoperability^{with other system}) but this does **not** guarantees that the system is secure (e.g. due to design or implementation faults)

Knowledge”



E OF THE SYSTEM

- Real security (based on **dec** protected)

The **specifications** of the system are
OPEN / CLOSED

If they have specs open/closed, doesn't guarantee security by definition

- What are the **effects** of this principle on software?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **deep knowledge** (of the system to be protected)
- What are the **effects** of this principle **on software**?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- Real security is based on **knowledge** (if the system is protected)

The source code (i.e. computer instructions) used by the programmers to implement the system are

OPEN / CLOSED

doesn't guarantee security by definition

- What are the **effects** of this principle on software?
 - **Open Systems** vs. **Closed Systems**
 - **Open Source** vs. **Closed Source**

Knowledge”

I really prefer **OPEN** Source (e.g. it can be *inspected, verified, validated* etc.) but this does not guarantees that the system is secure

GE OF THE SYSTEM

The source code (i.e. computer instructions) used by the programmers to implement the system are

OPEN / CLOSED

- Real security is based on **data** protected
- What are the **effects** of this principle on software?
 - Open systems vs. Closed Systems
 - **Open Source** vs. Closed Source



I really prefer **OPEN** Source (e.g. it can be *inspected, verified, validated* etc.) but this does **not** guarantees that the system is secure

Even if it is theoretically possible (for me) to check all the code in a system (e.g. an open source Operating System like GNU/Linux), this is not practically possible (since it is made of millions of lines of source code)

Even if i have the source code of the program, it is difficult to know if the binary version of it contains a backdoor or malicious code that is different from the source code that is public available

- Real security based on protected
- What are the effects of this principle on software?
 - Open systems vs. Closed Systems
 - **Open Source** vs. Closed Source

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Management of Trust is fundamental in Cybersecurity and based on a lot of different stuff (example: choosing the browser based on history of vuln. fixing)

KNOWLEDGE OF THE SYSTEM

- Real security is based on knowledge (of the system to be protected)

- What are the **effects**

- **Open Systems**

- **Open Source v**

In most cases it is a matter of

Security is based on Knowledge about the system we want to secure

TRUST

→ baseline of CySec (define the allies and enemies)

→ more important than knowledge (because it is very difficult to know/verify/check every single detail of the system)

Do you prefer to trust a
community (i.e. Open Source) or
a company (i.e. Close Source)

The future in terms of guaranteeing the supply-chain is a game not based on verifying everything, it will be based a lot on certifications (made in-house or by third-party certifiers) to demonstrate to customer that you are implementing cybersecurity

Product of a Manufacture is based on Software supply chain (built on libraries, services, etc.): if something bad happens the responsibility is on the Manufacture (this is why they need to certificate them to build trust on Manufacture's product)

Nobody, now, create SW that is totally developer e/o designed in house (we use a lot of library, sw made by others, etc.): possible problem of the supply chain (example: (supply-chain attack) possible backdoor in our program that came from library developed by others)

Auto-Updates is a pro and cons:

- Pro: install automatically the fix for a detected vuln.
- Cons: install updates automatic with backdoors or other vuln.

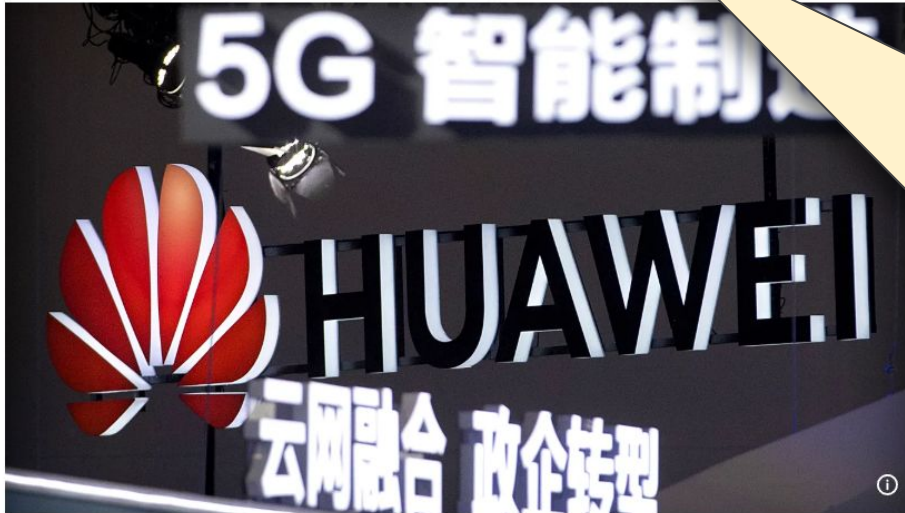
Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Eleven EU countries took 5G security measures to ban Huawei, ZTE

because they cannot go in deep on the software of these devices to check about vulnerabilities like backdoors (they restricted because they don't have trust on some companies: possible that China gather info through it (not verified))



«Eleven countries [...] have used legal powers to impose **restrictions** on telecom suppliers that are considered high-risk, such as Huawei and ZTE, for 5G network infrastructure, a European Commission spokesperson told Euronews»

Copyright Mark Schiefelbein/Copyright 2018 The AP. All rights reserved

By [Cynthia Kroet](#)

Published on 12/08/2024 - 16:37 GMT+2

Principle 3: on “Security = Knowledge”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KNOWLEDGE OF THE SYSTEM

- **Users' education** is a form of **knowledge**
- **Investing in education** is good but **not that simple**:

- education is **costly**

It is important but a few cares
about it (increasing in these day)

- **perception of security**: [lack of] importance of cybersecurity

- [wrong] **habits** (i.e. “we have always done it this way”)

- **ideological resistance** (i.e. “information anarchism”) (example: don't use passwords)

to win this battles:
use of regulations



- **cost of security** procedures (vs. “productive investments”)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Principle Important to the Prof

KERCKHOFFS'S PRINCIPLE

- “A cryptosystem should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

→ Algo that we use for encrypting the data, must be public (the only part that must be secret is the key, a small piece of information)

● “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883) It is against Security through obscurity

Cypher (cifrario)

It is a mechanism (i.e. hardware or
algorithm
software) used for encrypting information
with the aim to preserve confidentiality

(encryption to preserve this property)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

- “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

KERCKHOFFS'S PRINCIPLE

- “A **cryptosystem** should be secure **even if everything about the system, except the key, is public knowledge**” (1883)

SHANNON'S REFORMULATION



“The **enemy knows the system**”

not every secret of the system, but the design of the system

It is a much better way to base the sec on the selection of the proper algo, not on secrecy of the algo (maintaining private the key)

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURITY THROUGH OBSCURITY

Wrong: against Kerckhoffs
principle (algo secret)

- Means relying on the design or implementation secrecy as the main method of providing security

It is very difficult to maintain during the years

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

SECURITY THROUGH OBSCURITY

- Means relying on the **design** or **implementation secrecy** as the main method of providing security
- The **security through obscurity** is **WRONG**

Addendum: "Security of a Cryptosystem"



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

The **security through obscurity** is **WRONG**

- Does this means that I must **publish all the details about my system internals?**

When develop a new application, there are two separate path that you can choose to adopt encryption:

- 1) Use standards (Good Approach: study papers and choose the perfect one)
- 2) Create a new algo for encryption (Stupid Approach: will do something with a lot of vulnerabilities and errors, if you are not an expert)

The Core Flaws:

- Fragility: Once the "secret" is discovered, the security is 0. It has no backup plan.
- The "Script Kiddie" Problem: While it might hide you from basic automated bots, it won't stop a human attacker with basic tools like scanners or decompilers.
- Complacency: It often leads developers to ignore real vulnerabilities because they assume "no one will find it anyway."
- "Violates Kerckhoffs's Principle: True security should remain intact even if the attacker knows exactly how the system works, as long as they don't have your specific key."

Addendum: “Security of a Cryptosystem”



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Ask why and what you
are going to secure with
this security mechanism
(define a model)

The security through obscurity is **WRONG**

- Does this mean that I must **publish all the details about my system internals?**

• **NO**, it would be very stupid!



will decrease the cost for an attacker

The security must not be based on secrecy, but this doesn't mean that you have to publish all details (you need to increase the cost for attacker to breach the system or gather info)

When we build up a security mechanism, we are totally losing common sense and avoid only cosmetic security mechanism: the security mechanism is useful if it is possible to enforce it (users will try to bypass it)

Before disclosing information, evaluate whether it is truly necessary. Excessive disclosure fuels OSINT (Open Source Intelligence), allowing attackers to gather the data needed to refine and launch a more effective attack.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

CyberSecurity

Gabriele D'Angelo

<g.dangelo@unibo.it>

<https://www.unibo.it/sitoweb/g.dangelo/en>

www.unibo.it

Attributions and Notes



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

- These slides are partially based on the work of Prof. **Renzo Davoli**
- **Ludovico Gardenghi** has worked on a previous version of these slides
- I wish to **thank all of them**, **any errors that remain are my sole responsibility**